

ASSIGNMENT 4

Q1. Write a program to implement single Link List (with function insertion at first node, insertion at last node, deletion at first node, deletion at last node, insertion and deletion at middle node and traversing of link list).

```
#include <iostream>
using namespace std;

struct Node
{
    int data;
    Node *next;
};

Node *head = NULL;

void insertFirst()
{
    int x;

    cout << "Enter value: ";
    cin >> x;

    Node *newNode = new Node;

    newNode->data = x;
    newNode->next = head;

    head = newNode;

    cout << "Node inserted at first\n";
}

void insertLast()
{
    int x;

    cout << "Enter value: ";
    cin >> x;

    Node *newNode = new Node;

    newNode->data = x;
```

```

newNode->next = NULL;

if (head == NULL)
{
    head = newNode;
}
else
{
    Node *temp = head;

    while (temp->next != NULL)
    {
        temp = temp->next;
    }

    temp->next = newNode;
}

cout << "Node inserted at last\n";
}

void deleteFirst()
{
    if (head == NULL)
    {
        cout << "List is empty\n";
        return;
    }

    Node *temp = head;

    head = head->next;

    delete temp;

    cout << "First node deleted\n";
}

void deleteLast()
{
    if (head == NULL)
    {
        cout << "List is empty\n";
        return;
    }

```

```

    }

    if (head->next == NULL)
    {
        delete head;
        head = NULL;
        cout << "Last node deleted\n";
        return;
    }

    Node *temp = head;

    while (temp->next->next != NULL)
    {
        temp = temp->next;
    }

    delete temp->next;
    temp->next = NULL;

    cout << "Last node deleted\n";
}

void insertMiddle()
{
    int x, pos;

    cout << "Enter value: ";
    cin >> x;

    cout << "Enter position: ";
    cin >> pos;

    Node *newNode = new Node;

    newNode->data = x;

    if (pos == 1)
    {
        newNode->next = head;
        head = newNode;
        return;
    }

```

```

Node *temp = head;

for (int i = 1; i < pos - 1; i++)
{
    if (temp == NULL)
    {
        cout << "Invalid position\n";
        delete newNode;
        return;
    }

    temp = temp->next;
}

if (temp == NULL)
{
    cout << "Invalid position\n";
    delete newNode;
    return;
}

newNode->next = temp->next;
temp->next = newNode;

cout << "Node inserted\n";
}

void deleteMiddle()
{
    int pos;

    cout << "Enter position: ";
    cin >> pos;

    if (head == NULL)
    {
        cout << "List is empty\n";
        return;
    }

    if (pos == 1)
    {
        Node *temp = head;
        head = head->next;
    }
}

```

```

        delete temp;

        cout << "Node deleted\n";
        return;
    }

    Node *temp = head;

    for (int i = 1; i < pos - 1; i++)
    {
        if (temp == NULL)
        {
            cout << "Invalid position\n";
            return;
        }

        temp = temp->next;
    }

    if (temp == NULL || temp->next == NULL)
    {
        cout << "Invalid position\n";
        return;
    }

    Node *del = temp->next;

    temp->next = del->next;

    delete del;

    cout << "Node deleted\n";
}

void display()
{
    if (head == NULL)
    {
        cout << "List is empty\n";
        return;
    }

    Node *temp = head;

```

```

    cout << "Linked List: ";

    while (temp != NULL)
    {
        cout << temp->data << " ";
        temp = temp->next;
    }

    cout << endl;
}

int main()
{
    int choice;

    do
    {
        cout << "\n MENU \n";
        cout << "1. Insert at First\n";
        cout << "2. Insert at Last\n";
        cout << "3. Delete at First\n";
        cout << "4. Delete at Last\n";
        cout << "5. Insert at Middle\n";
        cout << "6. Delete at Middle\n";
        cout << "7. Display / Traverse\n";
        cout << "8. Exit\n";

        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice)
        {
            case 1:
                insertFirst();
                break;

            case 2:
                insertLast();
                break;

            case 3:
                deleteFirst();
                break;

```

```
case 4:
    deleteLast();
    break;

case 5:
    insertMiddle();
    break;

case 6:
    deleteMiddle();
    break;

case 7:
    display();
    break;

case 8:
    cout << "Program ended\n";
    break;

default:
    cout << "Wrong choice\n";
    }

} while (choice != 8);

return 0;
}
```

Terminal

Code + ▾ □ □ ...

```
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
```

MENU

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Enter your choice: 1

Enter value: 10

Node inserted at first

MENU

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Enter your choice: 2

Enter value: 20

Node inserted at last

Terminal

Code + ▾ □ 🗑️ ...

```
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
```

MENU

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Enter your choice: 2

Enter value: 30

Node inserted at last

MENU

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Enter your choice: 7

Linked List: 10 20 30

Terminal

Code + ▾ □ 🗑️ ...

```
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
```

MENU

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Enter your choice: 5

Enter value: 15

Enter position: 2

Node inserted

MENU

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Enter your choice: 6

Enter position: 3

Node deleted

Terminal

Code + ▾ □ □ ☒ ...

```
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
```

MENU

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Enter your choice: 7

Linked List: 10 15 30

MENU

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Enter your choice: 3

First node deleted

Terminal

Code + - [] [] ...

```
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
```

MENU

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Enter your choice: 10

Wrong choice

MENU

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Enter your choice: 7

Linked List: 15 30

Terminal

Code + ▾ □ 🗑️ ...

```
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
```

MENU

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Enter your choice: 4

Last node deleted

MENU

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Enter your choice: 7

Linked List: 15

MENU

Terminal

Code + ▾ 📄 🗑️ ...

```
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
```

7. Display / Traverse

8. Exit

Enter your choice: 7

Linked List: 15

MENU

1. Insert at First

2. Insert at Last

3. Delete at First

4. Delete at Last

5. Insert at Middle

6. Delete at Middle

7. Display / Traverse

8. Exit

Enter your choice: 8

Program ended

PS C:\Users\jainm\OneDrive\Desktop\dsa lab> █

Q2. Write a program to implement Doubly Link List (with function insertion at first node, insertion at last node, deletion at first node, deletion at last node, insertion and deletion at middle node and traversing of link list).

```
#include <iostream>
using namespace std;

struct Node
{
    int data;
    Node *prev;
    Node *next;
};

Node *head = NULL;

void insertFirst()
{
    int x;

    cout << "Enter value: ";
    cin >> x;

    Node *newNode = new Node;

    newNode->data = x;
    newNode->prev = NULL;
    newNode->next = head;

    if (head != NULL)
    {
        head->prev = newNode;
    }

    head = newNode;

    cout << "Node inserted at first\n";
}

void insertLast()
{
    int x;

    cout << "Enter value: ";
```

```
cin >> x;
```

```
Node *newNode = new Node;
```

```
newNode->data = x;
```

```
newNode->next = NULL;
```

```
if (head == NULL)
```

```
{
```

```
    newNode->prev = NULL;
```

```
    head = newNode;
```

```
}
```

```
else
```

```
{
```

```
    Node *temp = head;
```

```
    while (temp->next != NULL)
```

```
    {
```

```
        temp = temp->next;
```

```
    }
```

```
    temp->next = newNode;
```

```
    newNode->prev = temp;
```

```
}
```

```
cout << "Node inserted at last\n";
```

```
}
```

```
void deleteFirst()
```

```
{
```

```
    if (head == NULL)
```

```
    {
```

```
        cout << "List is empty\n";
```

```
        return;
```

```
    }
```

```
Node *temp = head;
```

```
head = head->next;
```

```
if (head != NULL)
```

```
{
```

```
    head->prev = NULL;
```

```
}
```

```

delete temp;

cout << "First node deleted\n";
}

void deleteLast()
{
    if (head == NULL)
    {
        cout << "List is empty\n";
        return;
    }

    if (head->next == NULL)
    {
        delete head;
        head = NULL;

        cout << "Last node deleted\n";
        return;
    }

    Node *temp = head;

    while (temp->next != NULL)
    {
        temp = temp->next;
    }

    temp->prev->next = NULL;

    delete temp;

    cout << "Last node deleted\n";
}

void insertMiddle()
{
    int x, pos;

    cout << "Enter value: ";
    cin >> x;

```

```

cout << "Enter position: ";
cin >> pos;

Node *newNode = new Node;

newNode->data = x;

if (pos == 1)
{
    newNode->prev = NULL;
    newNode->next = head;

    if (head != NULL)
    {
        head->prev = newNode;
    }

    head = newNode;

    cout << "Node inserted\n";
    return;
}

Node *temp = head;

for (int i = 1; i < pos - 1; i++)
{
    if (temp == NULL)
    {
        cout << "Invalid position\n";
        delete newNode;
        return;
    }

    temp = temp->next;
}

if (temp == NULL)
{
    cout << "Invalid position\n";
    delete newNode;
    return;
}

```

```

newNode->next = temp->next;
newNode->prev = temp;

if (temp->next != NULL)
{
    temp->next->prev = newNode;
}

temp->next = newNode;

cout << "Node inserted\n";
}

void deleteMiddle()
{
    int pos;

    cout << "Enter position: ";
    cin >> pos;

    if (head == NULL)
    {
        cout << "List is empty\n";
        return;
    }

    if (pos == 1)
    {
        Node *temp = head;

        head = head->next;

        if (head != NULL)
        {
            head->prev = NULL;
        }

        delete temp;

        cout << "Node deleted\n";
        return;
    }

    Node *temp = head;

```

```

for (int i = 1; i < pos; i++)
{
    if (temp == NULL)
    {
        cout << "Invalid position\n";
        return;
    }

    temp = temp->next;
}

if (temp == NULL)
{
    cout << "Invalid position\n";
    return;
}

if (temp->next != NULL)
{
    temp->next->prev = temp->prev;
}

if (temp->prev != NULL)
{
    temp->prev->next = temp->next;
}

delete temp;

cout << "Node deleted\n";
}

void display()
{
    if (head == NULL)
    {
        cout << "List is empty\n";
        return;
    }

    Node *temp = head;

    cout << "Doubly Linked List: ";

```

```

while (temp != NULL)
{
    cout << temp->data << " ";
    temp = temp->next;
}

cout << endl;
}

int main()
{
    int choice;

    do
    {
        cout << "\nMENU \n";
        cout << "1. Insert at First\n";
        cout << "2. Insert at Last\n";
        cout << "3. Delete at First\n";
        cout << "4. Delete at Last\n";
        cout << "5. Insert at Middle\n";
        cout << "6. Delete at Middle\n";
        cout << "7. Display / Traverse\n";
        cout << "8. Exit\n";

        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice)
        {
            case 1:
                insertFirst();
                break;

            case 2:
                insertLast();
                break;

            case 3:
                deleteFirst();
                break;

            case 4:

```

```
        deleteLast();
        break;

    case 5:
        insertMiddle();
        break;

    case 6:
        deleteMiddle();
        break;

    case 7:
        display();
        break;

    case 8:
        cout << "Program ended\n";
        break;

    default:
        cout << "Wrong choice\n";
    }

} while (choice != 8);

return 0;
}
```

Terminal

Code + - [] [] ...

```
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
```

MENU

```
1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit
Enter your choice: 1
Enter value: 10
Node inserted at first
```

MENU

```
1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit
Enter your choice: 2
Enter value: 20
Node inserted at last
```

Terminal

Code + ▾ 📄 🗑️ ...

```
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
```

MENU

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Enter your choice: 2

Enter value: 30

Node inserted at last

MENU

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Enter your choice: 7

Doubly Linked List: 10 20 30

Terminal

Code + ▾ □ 🗑️ ...

```
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
```

MENU

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Enter your choice: 5

Enter value: 15

Enter position: 2

Node inserted

MENU

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Enter your choice: 7

Doubly Linked List: 10 15 20 30

Terminal

Code + ▾ 📄 🗑️ ...

```
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
```

MENU

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Enter your choice: 6

Enter position: 3

Node deleted

MENU

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Enter your choice: 7

Doubly Linked List: 10 15 30

MENU

Terminal

Code + - [] [] ...

```
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
```

MENU

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Enter your choice: 3

First node deleted

MENU

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Enter your choice: 7

Doubly Linked List: 15 30

Terminal

Code + - [] [] ...

```
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
```

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Enter your choice: 4

Last node deleted

MENU

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Enter your choice: 7

Doubly Linked List: 15

MENU

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Enter your choice: 8

Program ended

Q3. Write a program to implement Circular Doubly Link List (with function insertion at first node, insertion at last node, deletion at first node, deletion at last node, insertion and deletion at middle node and traversing of link list).

```
#include <iostream>
using namespace std;

struct Node
{
    int data;
    Node *prev;
    Node *next;
};

Node *head = NULL;

void insertFirst()
{
    int x;

    cout << "Enter value: ";
    cin >> x;

    Node *newNode = new Node;

    newNode->data = x;

    if (head == NULL)
    {
        newNode->next = newNode;
        newNode->prev = newNode;
        head = newNode;
    }
    else
    {
        Node *last = head->prev;

        newNode->next = head;
        newNode->prev = last;

        last->next = newNode;
        head->prev = newNode;

        head = newNode;
    }
}
```

```

    }

    cout << "Node inserted at first\n";
}

void insertLast()
{
    int x;

    cout << "Enter value: ";
    cin >> x;

    Node *newNode = new Node;

    newNode->data = x;

    if (head == NULL)
    {
        newNode->next = newNode;
        newNode->prev = newNode;
        head = newNode;
    }
    else
    {
        Node *last = head->prev;

        newNode->next = head;
        newNode->prev = last;

        last->next = newNode;
        head->prev = newNode;
    }

    cout << "Node inserted at last\n";
}

void deleteFirst()
{
    if (head == NULL)
    {
        cout << "List is empty\n";
        return;
    }
}

```

```

if (head->next == head)
{
    delete head;
    head = NULL;
}
else
{
    Node *last = head->prev;
    Node *temp = head;

    head = head->next;

    head->prev = last;
    last->next = head;

    delete temp;
}

cout << "First node deleted\n";
}

void deleteLast()
{
    if (head == NULL)
    {
        cout << "List is empty\n";
        return;
    }

    if (head->next == head)
    {
        delete head;
        head = NULL;
    }
    else
    {
        Node *last = head->prev;
        Node *secondLast = last->prev;

        secondLast->next = head;
        head->prev = secondLast;

        delete last;
    }
}

```

```
    cout << "Last node deleted\n";  
}
```

```
void insertMiddle()
```

```
{  
    int x, pos;
```

```
    cout << "Enter value: ";  
    cin >> x;
```

```
    cout << "Enter position: ";  
    cin >> pos;
```

```
    Node *newNode = new Node;
```

```
    newNode->data = x;
```

```
    if (head == NULL)  
    {  
        cout << "List is empty\n";  
        delete newNode;  
        return;  
    }
```

```
    if (pos == 1)  
    {  
        Node *last = head->prev;
```

```
        newNode->next = head;  
        newNode->prev = last;
```

```
        last->next = newNode;  
        head->prev = newNode;
```

```
        head = newNode;
```

```
        cout << "Node inserted\n";  
        return;  
    }
```

```
    Node *temp = head;
```

```
    for (int i = 1; i < pos - 1; i++)
```

```

{
    temp = temp->next;

    if (temp == head)
    {
        cout << "Invalid position\n";
        delete newNode;
        return;
    }
}

newNode->next = temp->next;
newNode->prev = temp;

temp->next->prev = newNode;
temp->next = newNode;

cout << "Node inserted\n";
}

void deleteMiddle()
{
    int pos;

    cout << "Enter position: ";
    cin >> pos;

    if (head == NULL)
    {
        cout << "List is empty\n";
        return;
    }

    if (pos == 1)
    {
        if (head->next == head)
        {
            delete head;
            head = NULL;
        }
        else
        {
            Node *last = head->prev;
            Node *temp = head;

```

```

        head = head->next;

        head->prev = last;
        last->next = head;

        delete temp;
    }

    cout << "Node deleted\n";
    return;
}

Node *temp = head;

for (int i = 1; i < pos; i++)
{
    temp = temp->next;

    if (temp == head)
    {
        cout << "Invalid position\n";
        return;
    }
}

temp->prev->next = temp->next;
temp->next->prev = temp->prev;

delete temp;

cout << "Node deleted\n";
}

void display()
{
    if (head == NULL)
    {
        cout << "List is empty\n";
        return;
    }

    Node *temp = head;

```

```

cout << "Circular Doubly Linked List: ";

do
{
    cout << temp->data << " ";
    temp = temp->next;
}
while (temp != head);

cout << endl;
}

int main()
{
    int choice;

    do
    {
        cout << "\nMENU\n";
        cout << "1. Insert at First\n";
        cout << "2. Insert at Last\n";
        cout << "3. Delete at First\n";
        cout << "4. Delete at Last\n";
        cout << "5. Insert at Middle\n";
        cout << "6. Delete at Middle\n";
        cout << "7. Display / Traverse\n";
        cout << "8. Exit\n";

        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice)
        {
            case 1:
                insertFirst();
                break;

            case 2:
                insertLast();
                break;

            case 3:
                deleteFirst();
                break;

```

```
case 4:
    deleteLast();
    break;

case 5:
    insertMiddle();
    break;

case 6:
    deleteMiddle();
    break;

case 7:
    display();
    break;

case 8:
    cout << "Program ended\n";
    break;

default:
    cout << "Wrong choice\n";
}

} while (choice != 8);

return 0;
}
```

Terminal

Code + - [] [] ...

```
● PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab\" ; if ($?) { g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\tempCodeRunnerFile }
```

MENU

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Enter your choice: 1

Enter value: 10

Node inserted at first

MENU

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Enter your choice: 2

Enter value: 20

Node inserted at last

Terminal

Code + ▾ □ □ ...

```
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\d
```

MENU

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Enter your choice: 2

Enter value: 30

Node inserted at last

MENU

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Enter your choice: 7

Circular Doubly Linked List: 10 20 30

MENU

Terminal

Code + - [] [] ...

```
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab"
MENU
```

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Enter your choice: 4

Last node deleted

MENU

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Enter your choice: 7

Circular Doubly Linked List: 15

MENU

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Enter your choice: 8

Program ended

```
PS C:\Users\jainm\OneDrive\Desktop\dsa lab>
```

Terminal

Code + ▾ □ 🗑️ ...

```
PS C:\Users\jainm\OneDrive\Desktop\dsa lab> cd "c:\Users\jainm\OneDrive\Desktop\dsa lab"
MENU
```

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Enter your choice: 5

Enter value: 15

Enter position: 2

Node inserted

MENU

1. Insert at First
2. Insert at Last
3. Delete at First
4. Delete at Last
5. Insert at Middle
6. Delete at Middle
7. Display / Traverse
8. Exit

Enter your choice: 7

Circular Doubly Linked List: 10 15 20 30